ACADEMIC CENTRAL

PRACTICE EXAMINATION

NC(V) LEVEL 3

# INTRODUCTION TO ROBOTICS

## PAPER 2 · PRACTICE C · MARKING GUIDELINE

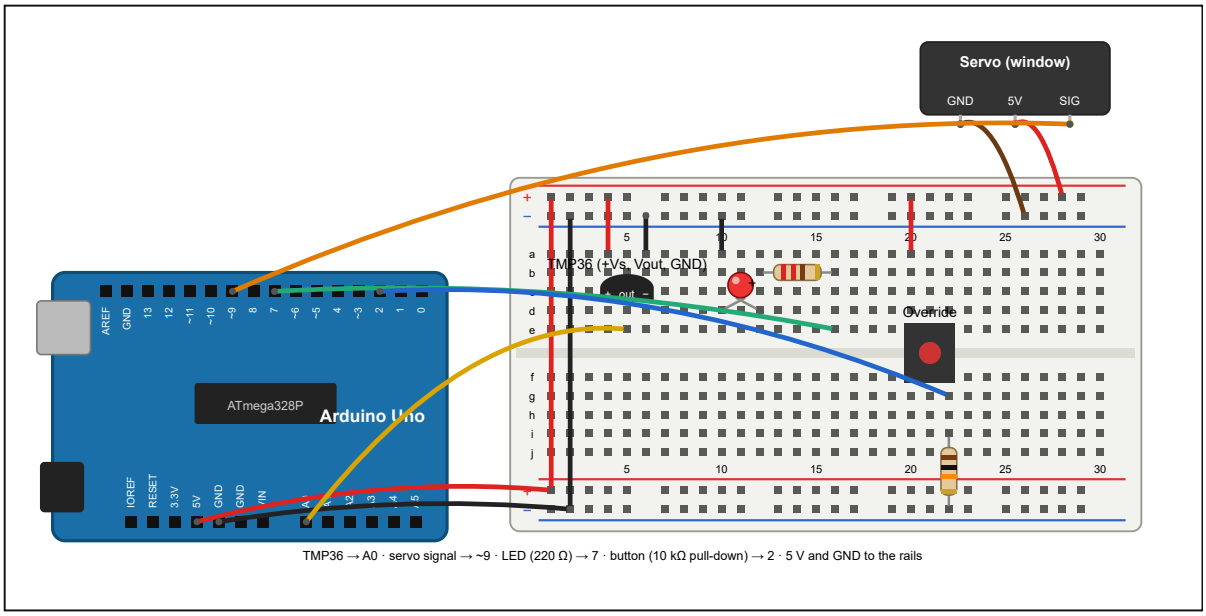
**MARKS: 80**

**Marking guideline for the Academic Central NC(V) Level 3 Introduction to Robotics paper 2 · practice c.**

Written by Academic Central for practice, following the NC(V) subject guidelines for NC(V) Level 3. It is not a Department of Higher Education and Training marking guideline.

**How to mark:** each ✓ is one mark. Practical work is not marked strictly on syntax. Give marks for correct logic, correct pins and connections, sensible comments and neat work. Accept other pin choices, layouts and program structures that work.

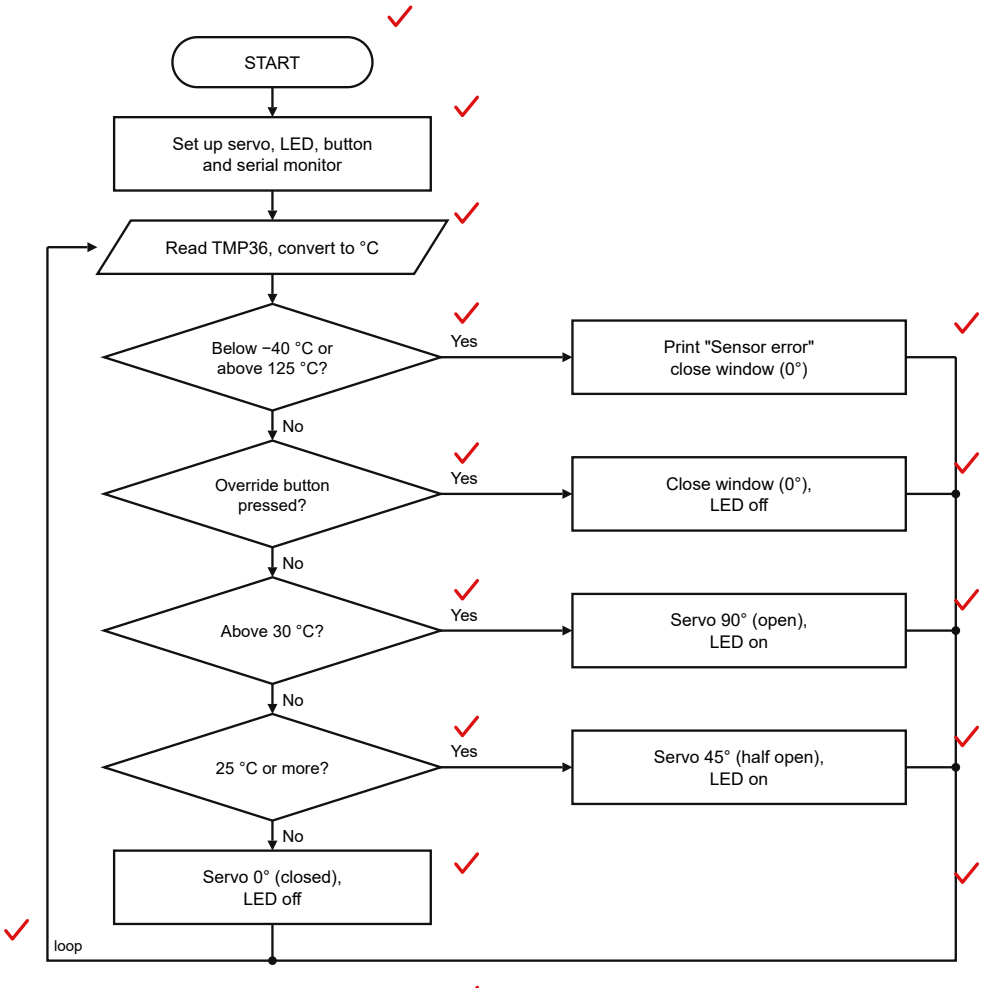
## QUESTION 1




**Example of correct wiring (other neat, correct layouts and pin choices get full marks)**

**1.1** TMP36: +Vs to 5 V ✓, GND to GND ✓, Vout to A0 ✓. Servo: signal (orange / yellow) to PWM pin 9 ✓, red to 5 V ✓, brown / black to GND ✓. LED: anode through the 220 Ω resistor ✓✓ to a digital pin (e.g. 7), cathode to GND ✓. Button: one side to 5 V ✓, other side to a digital pin (e.g. 2) ✓ with the 10 kΩ pull-down resistor from that pin to GND ✓. 5 V and GND rails from the Arduino to the breadboard ✓; neat, no short circuits ✓. (−1 for each wrong connection, −2 for any short circuit) (14)

**1.2**




(Correct flowchart symbols; marks as ticked: start, set-up, reading and converting the temperature (2), sensor-error check with its action (2), override check with its action (2), above 30 °C (2), 25 °C or more (2), closed position, loop back)

(14)

**1.3** Circuit saved and printed / presented ✓; flowchart saved and printed / presented ✓ (2)

**[30]**

## QUESTION 2

**2.1** Servo library and object ✓; attach to pin 9 ✓; LED and button pin modes ✓; Serial.begin ✓; analogRead(A0) ✓; voltage conversion ✓; temperature conversion ✓; error condition ✓ with "Sensor error" and window closed ✓; override closes the window ✓ and switches the LED off ✓; above 30 °C → 90° ✓; 25–30 °C → 45° ✓; below 25 °C → 0° ✓; servo.write() with the chosen angle ✓; LED on only when open ✓✓; useful comments ✓, e.g. (18)

```
#include <Servo.h>           // library for the servo motor
Servo window;                  // the roof-window servo
const int sensorPin = A0;      // TMP36 output
const int ledPin = 7;          // red LED: window open
const int buttonPin = 2;       // override button, HIGH when pressed

void setup() {
  window.attach(9);            // servo signal on PWM pin 9
  pinMode(ledPin, OUTPUT);
  pinMode(buttonPin, INPUT);   // external 10 kΩ pull-down resistor
```

```
  Serial.begin(9600);
  window.write(0);             // start with the window closed
}

void loop() {
  int reading = analogRead(sensorPin);           // 0 to 1023
  float voltage = reading * 5.0 / 1023;          // change to volts
  float temperature = (voltage - 0.5) * 100;     // change to °C
  Serial.println(temperature);
  int angle;

  if (temperature < -40 || temperature > 125) {  // impossible reading
    Serial.println("Sensor error");
    angle = 0;
  } else if (digitalRead(buttonPin) == HIGH) {   // override: keep closed
    angle = 0;
  } else if (temperature > 30) {
    angle = 90;                                   // fully open
  } else if (temperature >= 25) {
    angle = 45;                                   // half open
  } else {
    angle = 0;                                    // closed
  }

  window.write(angle);
  if (angle > 0) digitalWrite(ledPin, HIGH);     // LED on when open
  else digitalWrite(ledPin, LOW);
  delay(500);                                     // read twice a second
}
```

**2.2** Program saved ✓ and printed / presented ✓ (2)

**[20]**

## QUESTION 3

**3.1** After the 5 s steady phase ✓, a loop ✓ that repeats 3 times ✓ (or 6 half-second steps): WALK LED off ✓, delay 500 ms ✓, WALK LED on ✓, delay 500 ms ✓; then WALK and car red off ✓ and car green on ✓ (9)

**3.2** const int buzzer = 8; ✓ pinMode(buzzer, OUTPUT); ✓ buzzer switched on ✓ and off ✓ in a loop ✓ during the steady WALK phase ✓ and during the flashing phase ✓; buzzer silent when WALK is off ✓; total WALK time still correct ✓ (9)

**3.3** A variable to store the time of the last crossing ✓ (unsigned long ✓); set to millis() at the end of crossing() ✓✓; the if in loop() also checks millis() − lastCrossing ✓ ≥ 10 000 ✓ using AND (&&) ✓; cars stay green during that time ✓; logic correct ✓. (A delay(10000) after the crossing earns a maximum of 6 marks, because the program cannot do anything else during the delay.) (9)

```
const int buzzer = 8;               // 3.2
unsigned long lastCrossing = 0;     // 3.3: time the last crossing ended

// in setup():  pinMode(buzzer, OUTPUT);

void loop() {
  if (digitalRead(button) == LOW && millis() - lastCrossing >= 10000) {  // 3.3
    crossing();
  }
}
```

```
void crossing() {
  digitalWrite(carGreen, LOW);
  digitalWrite(carYellow, HIGH);
  delay(2000);
  digitalWrite(carYellow, LOW);
  digitalWrite(carRed, HIGH);
  digitalWrite(walkLed, HIGH);                // people may cross
  for (int i = 0; i < 10; i++) {              // 10 x 0,5 s = 5 s steady, with beeps (3.2)
    digitalWrite(buzzer, HIGH); delay(250);
    digitalWrite(buzzer, LOW);  delay(250);
  }
  for (int i = 0; i < 3; i++) {               // 3 s of flashing (3.1)
    digitalWrite(walkLed, LOW); delay(500);   // WALK off, buzzer silent
    digitalWrite(walkLed, HIGH);              // WALK on, beep
    digitalWrite(buzzer, HIGH); delay(250);
    digitalWrite(buzzer, LOW);  delay(250);
  }
  digitalWrite(walkLed, LOW);
  digitalWrite(carRed, LOW);
  digitalWrite(carGreen, HIGH);               // cars may go again
  lastCrossing = millis();                    // 3.3
}
```

**3.4** Final program saved ✓, printed / presented ✓, with comments showing the changes ✓ (3)

**[30]**

**TOTAL: 80**

**END OF MARKING GUIDELINE**